VIETTEL DIGITAL TALENT 2026

DATA ENGINEER

Streaming Processing in BigData Platform

A comprehensive study of real-time data pipelines
using Apache Kafka and Apache Flink

Mentor : Nguyen Minh Hieu

Student : Ngo Quang Hieu

July 2, 2026

Contents

List of Figures

List of Tables

List of Listings

Chapter 1

Introduction

1.1 Background and Motivation

The digital economy has fundamentally transformed how data is generated and consumed. Modern applications — social media platforms, e-commerce systems, IoT devices, and music streaming services — produce enormous volumes of events continuously and at high velocity. According to industry estimates, the global volume of data generated daily exceeds 2.5 quintillion bytes, and a significant portion of this data has value only if it is acted upon immediately.

Traditional **batch processing** pipelines, built on tools such as Apache Hadoop MapReduce or scheduled SQL jobs, collect data over a fixed window (hourly, daily) and process it in bulk. While effective for historical reporting, batch pipelines suffer from inherent latency: insights derived from yesterday's data cannot drive today's real-time decisions. A music streaming platform, for instance, cannot recommend songs based on what a user listened to an hour ago — the recommendation must be computed within seconds of the current play event.

Stream processing addresses this gap by treating data as a continuous, unbounded flow of events and computing results incrementally as each event arrives. This paradigm enables:

- **Real-time personalisation** — recommendations, rankings, and alerts computed within milliseconds of an event.
- **Operational intelligence** — dashboards and anomaly detection systems that reflect the current state of the system, not a stale snapshot.
- **Event-driven architectures** — microservices that react to changes in a shared event log rather than polling a database.

- **Fault-tolerant pipelines** — systems that can recover from failures without reprocessing hours of backlogged data.

Two foundational technologies underpin modern streaming platforms: a **distributed message queue** (Apache Kafka) that reliably captures and transports events at scale, and a **stream-processing engine** (Apache Flink or Apache Spark) that transforms those events in real time. This report examines both layers in depth, and demonstrates their use in a concrete end-to-end project — *Streamify* — that processes simulated music streaming events from ingestion through to interactive business intelligence dashboards.

1.2 Problem Statement

A music streaming platform generates three distinct categories of events in real time:

- **Song-play events** (`listen_events`) — fired each time a user starts playing a track, carrying metadata such as artist name, song title, duration, user location, and session information.
- **Page-view events** (`page_view_events`) — fired on every navigation action (e.g. visiting the home page, search results, or an artist profile), capturing HTTP method, status code, and user context.
- **Authentication events** (`auth_events`) — fired on login and logout, recording session validity and subscription level.

To illustrate the scale of these events, Table 1.1 shows estimated event volumes under a modest 10,000 concurrent user simulation:

Table 1.1: Estimated event rates for a 10,000 concurrent user simulation.

Event type	**Avg. rate (events/s)**	**Volume per hour**
`listen_events`	∼300	∼1.1 million
`page_view_events`	∼150	∼540,000
`auth_events`	∼50	∼180,000
Total	∼500	∼1.8 million

Processing these events in batch is insufficient for four reasons:

1. **Velocity** — at hundreds of events per second, buffering them for even one hour means acting on data that is already stale. A song that trended at 9 pm cannot drive the 9 pm featured-artists chart if the pipeline only runs at midnight.

2. **Volume** — 1.8 million events per hour accumulate to over 43 million per day. A batch job processing an entire day's events requires significant infrastructure and introduces a latency proportional to the window size.

3. **Variety** — the three event types have different schemas, require different transformations (timestamp normalisation, string decoding, partition column derivation), and must eventually be joined with dimension data (users, songs, artists) for analytics. Handling schema heterogeneity in batch SQL is cumbersome; a streaming job can apply per-topic transformations in parallel.

4. **Reliability** — any processing outage must not result in data loss. A batch job that crashes mid-run may leave partial results or require a full re-run. A streaming pipeline with offset-based checkpointing can resume exactly where it stopped, processing only the events that were not yet committed.

The core challenge addressed in this report is therefore: *how to design and implement a fault-tolerant, low-latency streaming pipeline that ingests heterogeneous music events, transforms and enriches them in real time, and delivers clean analytical data to a cloud data warehouse — all without data loss or duplicate records.*

1.3 Objectives

This report and the accompanying project pursue the following objectives:

1. **Survey stream-processing concepts** — introduce the theoretical foundations of stream processing, including event time, watermarks, stateful computation, fault tolerance, and delivery semantics (at-most-once, at-least-once, exactly-once).

2. **Evaluate Apache Kafka as a message queue** — examine Kafka's architecture (brokers, topics, partitions, consumer groups), its evolution from ZooKeeper-based coordination to the self-managed KRaft consensus protocol, and its suitability as the event transport layer for high-throughput streaming pipelines.

3. **Compare Apache Spark and Apache Flink** — provide a detailed, side-by-side analysis of the two dominant open-source stream-processing engines, covering their execution models (micro-batch vs. true streaming), fault-tolerance mechanisms, state backends, and applicable use cases.

4. **Design and implement Streamify** — build a complete, containerised, end-to-end data pipeline that:

- simulates realistic music streaming events using Eventsim;
- transports events through Apache Kafka;
- processes and enriches them in real time with Apache Flink (PyFlink);
- writes clean records to a Snowflake cloud data warehouse;
- models the data into a star schema and a wide table using dbt;
- orchestrates daily dbt runs with Apache Airflow.

5. **Visualise insights** — connect Power BI to the star schema in Snowflake and build an interactive dashboard that surfaces key business metrics such as top artists, play counts by location, and subscription-tier distribution.

1.4 Scope

In Scope

- Design and implementation of the full streaming pipeline from event generation to BI dashboard.
- Theoretical coverage of Apache Kafka, Apache Spark Structured Streaming, and Apache Flink.
- Comparison of Spark micro-batch and continuous (RTM) execution modes against Flink's true-streaming model.
- Kafka's metadata evolution: ZooKeeper-based coordination vs. the KRaft self-managed consensus protocol.
- Data modelling with dbt (star schema + wide table) and orchestration with Apache Airflow.
- Local deployment using Docker Compose for full reproducibility.

Out of Scope

- Machine-learning inference on live event streams (e.g. real-time recommendation models).
- Multi-region or cloud-native Kubernetes deployment of the Flink or Kafka clusters.
- Production-grade SLA tuning, capacity planning, or cost optimisation.

- Schema evolution and backward/forward compatibility strategies for Kafka topics.
- Security hardening (TLS, SASL authentication, Snowflake private link).

1.5 Report Structure

The remainder of this report is organised as follows:

Chapter 2 — Message Queue: Apache Kafka

Introduces Apache Kafka as the event transport backbone of the pipeline. It explains why a durable message queue is necessary in distributed systems, walks through Kafka's core components (producers, brokers, topics, partitions, consumer groups), surveys common industry use cases, and traces Kafka's metadata architecture evolution from ZooKeeper to the modern KRaft protocol.

Chapter 3 — Streaming Processing Engines: Spark vs. Flink

Provides an in-depth comparison of the two leading open-source stream-processing frameworks. The chapter covers Apache Spark's micro-batch and continuous processing modes, then Apache Flink's true-streaming dataflow model. For each engine, the internal mechanisms — checkpointing, state storage, fault tolerance, and delivery semantics — are examined in detail. A side-by-side comparison table and a justification for choosing Flink in the Streamify project conclude the chapter.

Chapter 4 — Application: Streamify Pipeline

Presents the practical implementation of the Streamify project. It describes the full technology stack, the event simulation layer (Eventsim), the PyFlink streaming job that reads from Kafka and writes to Snowflake, the dbt data models that transform raw events into a star schema, and the Power BI dashboard that visualises the resulting business insights.

Chapter 2

Message Queue: Apache Kafka

2.1 Why Do We Need Kafka

2.1.1 The Problem with Direct Communication

Apache Kafka [4, 15] is a distributed event-streaming platform designed to decouple producers from consumers at scale. In a naive distributed system, services communicate directly: a producer service calls a consumer service's API whenever it has new data to share. This point-to-point architecture quickly breaks down as the system grows:

- **Tight coupling** — every producer must know the address, protocol, and schema of every consumer. Adding a new consumer requires modifying every upstream producer; removing one breaks others.
- **No buffering** — if a consumer is temporarily offline or slow, the producer either blocks (degrading its own performance) or drops messages (causing data loss).
- **No replay** — once a message is delivered, it is consumed and gone. If a downstream system fails mid-processing, there is no way to re-read the original event.
- **Synchronous bottleneck** — a synchronous request-response path cannot absorb sudden traffic spikes; a surge in producer activity cascades directly to consumers.
- **Fan-out complexity** — delivering the same event to multiple independent consumers (e.g. an analytics pipeline *and* a fraud detector) requires the producer to make N separate calls, one per consumer.

2.1.2 How Kafka Solves These Problems

Apache Kafka inserts a durable, distributed **commit log** between producers and consumers, decoupling them in time, space, and implementation:

- **Decoupling** — producers write to a named topic; consumers subscribe to that topic. Neither side knows about the other; new consumers can be added without touching producers.
- **Durable buffering** — records are persisted to disk and replicated across brokers. A consumer that goes offline for hours can catch up by reading from its last committed offset.
- **Replay** — Kafka retains records for a configurable window (hours, days, or forever with log compaction). Any consumer can re-read past events for reprocessing, debugging, or bootstrapping a new service.
- **Horizontal scalability** — topics are partitioned; each partition is an independent, ordered log. Adding partitions linearly scales both write throughput and read parallelism.
- **Fan-out for free** — multiple consumer groups can independently read the same topic at their own pace, each maintaining its own offset cursor, with zero overhead for the producer.
- **High durability** — a configurable replication factor ensures that data survives the failure of one or more brokers.

2.2 Architecture

Kafka's architecture revolves around a small set of primitives that compose into a highly scalable, fault-tolerant messaging backbone. Figure 2.1 shows the high-level cluster layout.

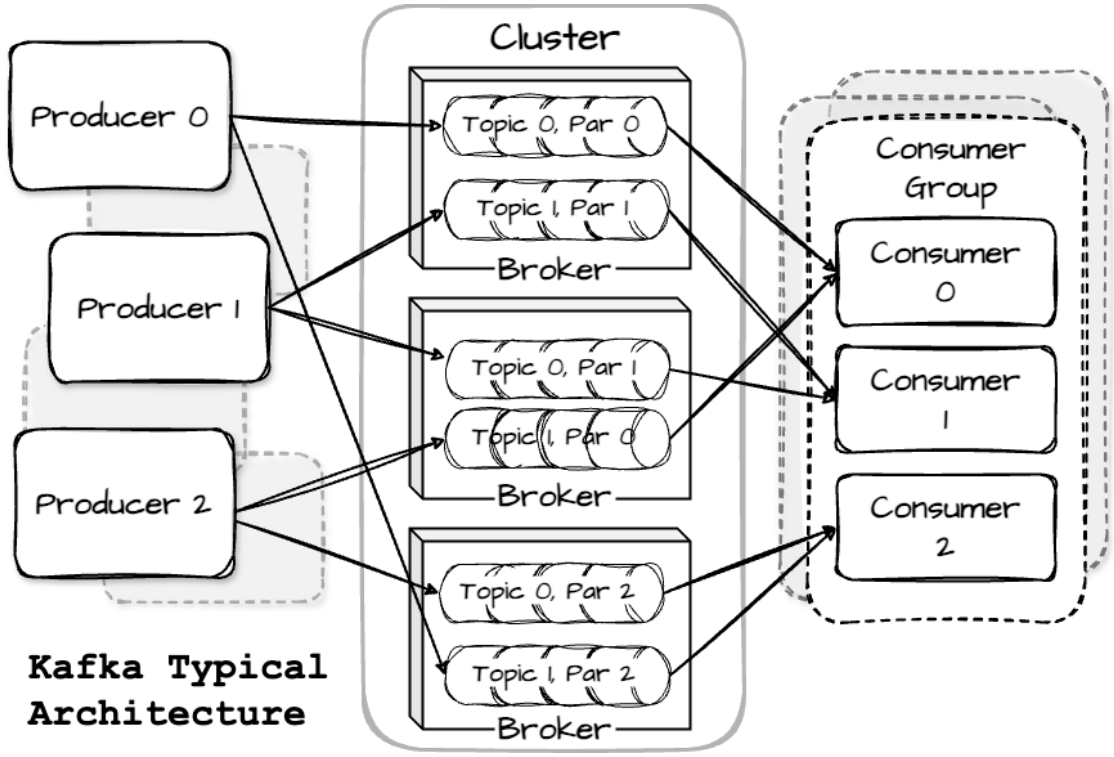

Figure 2.1: Apache Kafka cluster architecture: producers, brokers, topics, partitions, and consumer groups.

2.2.1 Producer

A **producer** is any client application that publishes records to one or more Kafka topics. The producer library handles the following automatically:

- **Partitioning** — the producer chooses a target partition for each record. If the record carries a *key*, the partition is determined by `hash(key) % numPartitions`, guaranteeing that all records with the same key land in the same partition (and are therefore totally ordered). Without a key, records are distributed round-robin across partitions for even load balancing.
- **Batching** — records destined for the same partition are accumulated in an in-memory buffer (`batch.size`) and sent together after a short delay (`linger.ms`), dramatically improving network throughput at the cost of a small, tunable latency.
- **Compression** — entire batches can be compressed with GZIP, Snappy, LZ4, or ZSTD before transmission, reducing both network bandwidth and broker disk usage.
- **Acknowledgement modes (`acks`)** — the producer waits for an acknowledgement from the broker before considering a write successful:

  - `acks=0`: fire-and-forget — maximum throughput, zero durability guarantee.
  - `acks=1`: leader acknowledges — fast, but data may be lost if the leader crashes before replicating.
  - `acks=all`: all in-sync replicas (ISR) acknowledge — strongest durability, moderate latency cost.

- **Idempotent producer** — when enabled (`enable.idempotence=true`), the broker deduplicates retried batches using a sequence number, providing exactly-once write semantics.

2.2.2 Broker

A **broker** is a Kafka server process responsible for persistently storing records and serving read/write requests from producers and consumers.

- Each broker stores a subset of the cluster's partition **replicas**. For every partition, one broker is the **leader** (handles all reads and writes); the remaining brokers in the ISR are **followers** that passively replicate from the leader.
- If a leader broker crashes or becomes unreachable, Kafka elects a new leader from the ISR set automatically — typically within seconds — with no manual intervention required.
- Records are written to a partition as an **append-only segment log** on local disk. Sequential I/O (rather than random seeks) is the primary reason Kafka achieves very high write throughput even on commodity spinning disks.
- A healthy production cluster typically runs **3–5 brokers**, with a replication factor of 3, tolerating up to 2 simultaneous broker failures without data loss.

2.2.3 Topic

A **topic** is a named, logical channel to which producers append records and from which consumers read. Think of it as a database table, except that:

- Records are **immutable** once appended — they cannot be updated or deleted (only expired).
- **Retention** controls how long records survive on disk. Time-based retention (e.g. 7 days) deletes old segments once they age out; size-based retention deletes once the topic exceeds a configured byte limit.

- **Log compaction** is an alternative mode that retains the latest record for each key indefinitely, making the topic behave like a key-value changelog — ideal for CDC (Change Data Capture) streams.

2.2.4 Partition

Each topic is divided into one or more **partitions**, which are the fundamental unit of parallelism and ordering in Kafka.

- A partition is an **ordered, immutable sequence** of records. Each record is assigned a monotonically increasing **offset** upon arrival, uniquely identifying its position within that partition.
- Ordering guarantees are *within* a partition only. If global ordering matters (e.g. all events for a single user), the application must route those events to the same partition via a consistent key.
- The number of partitions in a topic is the main knob for scaling: more partitions = more parallel writes by producers and more parallel reads by consumers. However, partitions cannot be reduced once created, and too many partitions increase leader-election overhead, so the partition count requires careful planning.
- Each partition replica is stored as a sequence of **segment files** on the broker's disk, with an index file for fast offset lookups.

2.2.5 Consumer and Consumer Group

A **consumer** reads records from a partition by *polling* the broker and tracking its current position via a **committed offset**.

- **Pull model** — consumers pull data at their own pace. Kafka never pushes records, so a slow consumer does not apply back-pressure to the broker or to other consumer groups reading the same topic.
- **Offset management** — the consumer commits the offset of the last record it has successfully processed to a special internal topic (`__consumer_offsets`). On restart, it resumes from the committed offset, preventing both data loss and duplicate processing. Commits can be *automatic* (periodic) or *manual* (after the application confirms success).

- **Consumer group** — a logical group of consumer instances that jointly consume a topic. Kafka assigns each partition to *exactly one* member of the group at any point in time, distributing load evenly. Multiple independent groups can consume the same topic simultaneously, each with its own offset cursor — this is how Kafka achieves fan-out without duplicating data.
- **Rebalancing** — when a consumer joins or leaves the group (or a partition count changes), the group coordinator triggers a rebalance that reassigns partitions among the active members. Modern Kafka clients support *cooperative rebalancing* to minimise disruption during the handover.
- **Parallelism rule** — the maximum useful parallelism for a consumer group equals the number of partitions in the topic; adding more consumers than partitions leaves the extra consumers idle.

2.3 Use Cases

Apache Kafka has become the de facto standard event backbone across a wide range of industries. Table 2.1 summarises the canonical use cases; the subsections below explain each pattern in more detail.

Table 2.1: Common Apache Kafka use cases.

Use Case	**Description**
Real-time event streaming	Collect click streams, IoT telemetry, or application logs and process them with Flink or Spark.
Microservice messaging	Decouple services with an async pub/sub channel instead of synchronous REST or gRPC calls.
Change Data Capture (CDC)	Stream database change events (via Debezium) to keep downstream systems in sync.
Log aggregation	Centralise logs from hundreds of servers into a single, queryable stream.
Metrics & monitoring	Route time-series metrics to alerting pipelines and observability stores.
Event sourcing	Use a compacted topic as the system of record; reconstruct application state by replaying events.
Stream ETL	Read raw events, transform and enrich them in flight (Flink/Spark), and land clean data in a warehouse.

2.3.1 Real-time Event Streaming

High-velocity event streams — user click sequences on an e-commerce site, GPS pings from a fleet of vehicles, sensor readings from industrial IoT devices — are a natural fit for Kafka. Events are published by producers (edge devices, application servers, SDKs) and consumed by one or more independent processing engines (Flink, Spark, or custom consumers) with millisecond ingestion latency. A key advantage over direct HTTP calls is **durability**: if the downstream processor restarts or falls behind, it replays events from its last committed offset rather than losing them. This pattern is the one used in Streamify, where Eventsim publishes music events to three Kafka topics and PyFlink consumes them for real-time processing.

2.3.2 Microservice Messaging

In a synchronous microservice architecture, service A calls service B's REST API directly. When service B is slow or unavailable, service A blocks or drops the request. Kafka replaces this point-to-point call with a shared topic: service A publishes an event and returns immediately; service B (and any other subscriber) processes it asynchronously at its own pace. This **temporal decoupling** eliminates cascading failures and allows services to be deployed, scaled, or replaced independently. Unlike traditional message queues (RabbitMQ, ActiveMQ), Kafka retains events after delivery, enabling late consumers to join the system and catch up on historical events.

2.3.3 Change Data Capture (CDC)

CDC is the practice of streaming every INSERT, UPDATE, and DELETE from a relational database into downstream systems in near real time. **Debezium**, the leading open-source CDC tool, connects to the database's replication log (PostgreSQL WAL, MySQL binlog, Oracle LogMiner) and publishes change events to Kafka topics. Consumers can then build materialised views, search indexes, or analytical aggregates that stay current within seconds of the source database change — without polling the database. Kafka's log compaction feature is particularly valuable here: a compacted topic retains the latest record per key, effectively functioning as a full replica of the current database state.

2.3.4 Log Aggregation

Distributed systems running hundreds of servers produce logs in disparate formats across many machines. Shipping these logs directly to a central store (Elasticsearch, Splunk, S3) creates tight coupling between log producers and the storage backend. Kafka acts as

a **log buffer**: lightweight agents (Filebeat, Fluentd) tail log files and publish entries to a Kafka topic; consumers (Logstash, Flink) parse, filter, and route them to storage. This decoupling means the log storage backend can be changed or scaled without touching the hundreds of log-producing services.

2.3.5 Metrics and Monitoring

Time-series metrics (CPU utilisation, request latency percentiles, error rates) are naturally event-shaped data. Publishing metrics to Kafka allows them to fan out to multiple consumers simultaneously: a Prometheus-compatible consumer for real-time alerting, a Flink job for anomaly detection, and an S3 consumer for long-term retention — all from a single Kafka topic, with no additional load on the metrics producer. Kafka's configurable retention window also enables retrospective analysis: if an alert fires at 3 am, the raw metric stream can be replayed from before the incident began.

2.3.6 Event Sourcing

In an event-sourced system, the authoritative state of an entity is not stored as a mutable database row but as an ordered sequence of immutable events in Kafka (e.g. `OrderPlaced`, `PaymentReceived`, `OrderShipped`). Application state is reconstructed on demand by replaying the event log. Kafka's **log compaction** mode retains the latest record for each key, providing an efficient snapshot of the current state without replaying the full history. This pattern provides a natural audit log, simplifies debugging (replay past events to reproduce bugs), and enables event-driven projections to be rebuilt at any time.

2.3.7 Stream ETL

Traditional ETL (Extract → Transform → Load) runs nightly batch jobs that move data from operational systems into data warehouses. Stream ETL (more accurately described as ELT) inverts this: events land in Kafka continuously, a stream processor (Flink or Spark) transforms and enriches them in real time, and clean records are written to the warehouse within seconds of the original event. This is the pattern used in Streamify: Eventsim → Kafka → PyFlink (enrich, deduplicate) → Snowflake (raw staging) → dbt (star schema) → Power BI. The result is a BI dashboard whose data is fresher than any nightly batch pipeline could achieve.

Chapter 3

Streaming Processing Engines: Spark vs Flink

3.1 Overview of Stream Processing

3.1.1 Batch vs. Stream Processing

Data processing systems can be broadly categorised by when computation occurs relative to data arrival:

- **Batch processing** — data is collected over a time window, stored, and then processed in a single large job. Systems like Apache Hadoop MapReduce and scheduled SQL pipelines fall into this category. Batch processing is well-suited to historical reports and large-scale transformations where latency is measured in hours or days.
- **Stream processing** — computation is performed *continuously* as each record arrives. Results are produced with latency measured in milliseconds to seconds. This model is essential for real-time dashboards, fraud detection, IoT monitoring, and any use case where acting on stale data is unacceptable.

3.1.2 Key Challenges in Stream Processing

Building a correct, efficient stream-processing system is significantly harder than batch processing, due to several inherent challenges:

- **Unbounded data** — the stream has no defined end; the system must process records indefinitely without accumulating unbounded state.

- **Out-of-order events** — network delays, retries, and clock skew mean that events frequently arrive out of their logical order. A system must decide when it is safe to emit a windowed result even if some late events may still be in transit.
- **Stateful computation** — aggregations, joins, and deduplication require maintaining keyed state across events. This state must be fault-tolerant and scalable.
- **Fault tolerance** — any node in the cluster can fail at any time. The system must recover to a consistent state without losing records or producing duplicates.
- **Delivery semantics** — depending on the use case, the system must guarantee *at-most-once* (may lose), *at-least-once* (may duplicate), or *exactly-once* (no loss, no duplicate) processing.

3.1.3 Two Leading Engines

This chapter examines the two most widely adopted open-source stream-processing frameworks:

Apache Spark Structured Streaming — extends Spark's batch DataFrame API to streaming data via a *micro-batch* execution model, with an experimental *continuous processing* (real-time) mode for lower latency.

Apache Flink — a purpose-built, true-streaming engine that processes events one at a time using a *dataflow graph* model, with first-class support for event time, watermarks, and exactly-once semantics via the Chandy-Lamport checkpointing algorithm.

3.2 Apache Spark

3.2.1 Overview

Apache Spark [5] is a unified, open-source analytics engine for large-scale data processing, originally developed at UC Berkeley's AMPLab in 2009 and donated to the Apache Software Foundation in 2010. Its central design goal is a *single programming model* — the DataFrame / Dataset API — that executes identically for batch jobs, streaming queries, machine-learning workloads (MLlib), and graph analytics (GraphX).

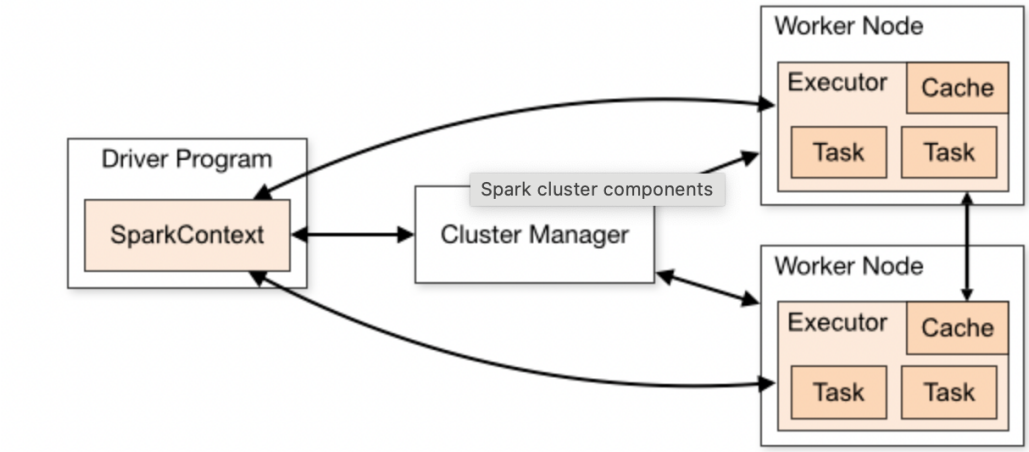

Figure 3.1: Apache Spark cluster architecture: Driver, Cluster Manager, and Executors.

Key internal components that make Spark fast:

- **Catalyst optimiser** — a rule-based and cost-based query planner that rewrites logical plans into optimal physical plans. It applies predicate pushdown, constant folding, join reordering, and many other transformations automatically.
- **Tungsten execution engine** — replaces JVM object representation with binary, off-heap memory layout and generates whole-stage bytecode at runtime (code generation), achieving near-native CPU efficiency.
- **RDD (Resilient Distributed Dataset)** — the foundational fault-tolerant abstraction. An RDD is a partitioned collection of records with a lineage graph that allows any lost partition to be recomputed from its parent RDDs. DataFrames and Datasets are typed views built on top of RDDs.

3.2.2 Spark Streaming: Micro-batch and Real-time Mode

Spark Structured Streaming extends the DataFrame API to unbounded data streams. The key insight is that a stream is treated as a *continuously appended, unbounded table*: each new trigger appends new rows, and the query always returns the same DataFrame-style result.

Micro-batch Mode

Figure 3.2: Spark Structured Streaming micro-batch execution model.

In micro-batch mode (the default), the driver divides time into discrete *trigger intervals* and processes all records that arrived in each interval as a single bounded batch:

1. The driver queries the source (e.g. Kafka) for new offsets since the previous trigger.
2. It constructs a bounded DataFrame containing those records (the *micro-batch*).
3. Catalyst optimises the query plan and Tungsten executes it across the Executor pool.
4. Results are written to the sink; source offsets are committed to the checkpoint store.
5. After the configured **trigger interval** (e.g. `processingTime="5 seconds"`), the cycle repeats.

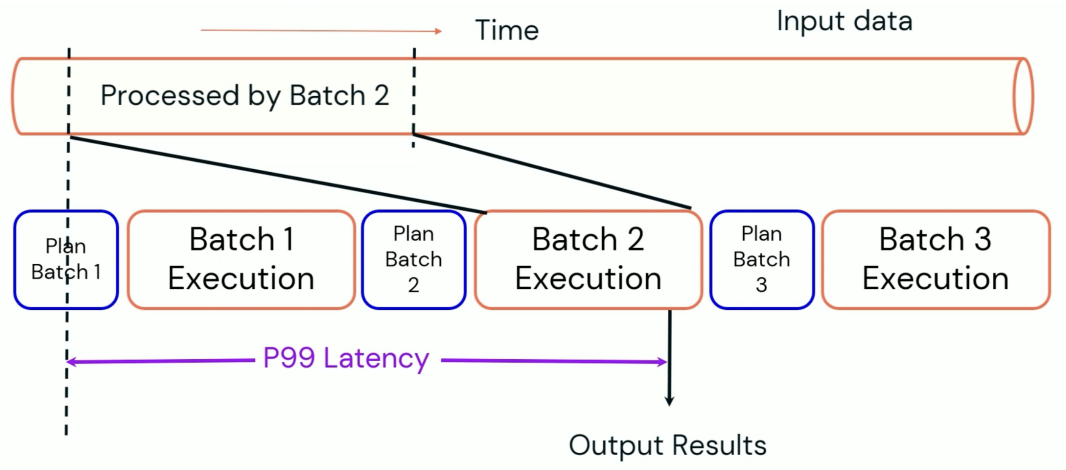

Figure 3.3: End-to-end latency profile in micro-batch mode: the trigger interval sets the latency floor.

The trigger interval is the primary latency knob: shorter intervals lower latency but increase scheduling overhead and reduce throughput per batch. In practice, sub-100 ms end-to-end latency is not achievable in this mode.

Real-time Mode (Continuous Mode) — New in Spark 4

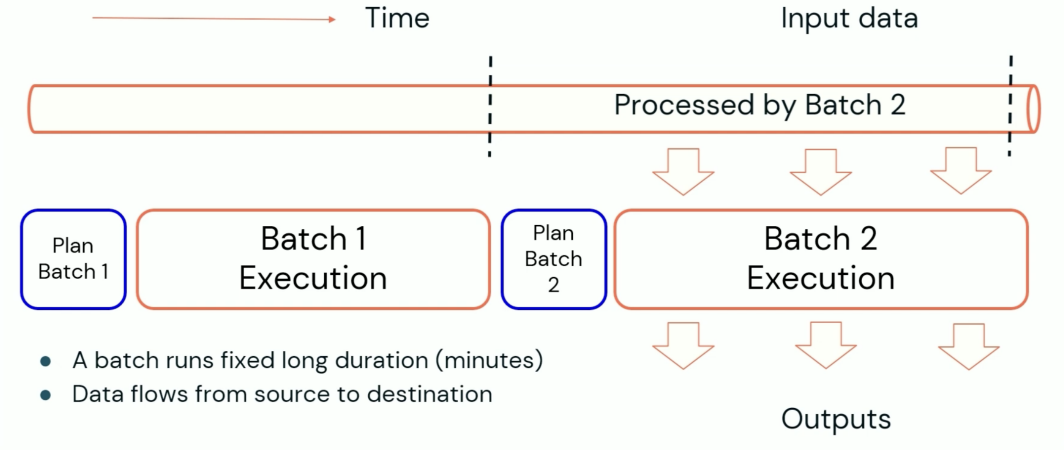

Figure 3.4: Spark Real-time Mode: data streams continuously through the operator pipeline within a long-running batch window.

Real-time Mode (also referred to as Continuous Mode) is a new execution mode introduced in **Spark 4 / Databricks Runtime**, announced at the Data + AI Summit. Rather than shrinking the micro-batch interval to reduce latency, it takes the opposite approach: batches run for a *long fixed window* (default 5 minutes), but data flows *continuously* through the operator pipeline inside that window — never buffered to the batch boundary.

The fundamental changes made to the Spark runtime to enable this:

1. **Concurrent stage scheduling** — in micro-batch mode, stages execute *sequentially* (stage n starts only after stage $n-1$ finishes). In real-time mode, all stages are scheduled *concurrently* from the start of the batch window, so a record flows from source to sink without waiting for upstream stages to complete a full sweep.

2. **Fresh source reads** — instead of reading a fixed offset range determined at the start of each batch, the source operator reads the *most recent available data* continuously as it arrives, ensuring records enter the pipeline with minimal ingestion delay.

3. **Streaming shuffle** — the shuffle layer (data exchange between stages) is redesigned to forward records immediately as they are produced, rather than materialising a full shuffle file at the end of a stage. This eliminates the per-stage synchronisation barrier that adds latency in micro-batch mode.

4. **Operator-level streaming** — aggregation and stateful operators (including the

new `TransformWithState` API) are modified to emit results incrementally as records arrive, rather than flushing outputs only at the end of each stage.

5. **Incremental state cleanup** — instead of scanning the entire state store at the end of every batch, state expiry is spread across the full batch window — a few state-store rows are checked after every processed record, keeping per-record latency flat regardless of state store size.

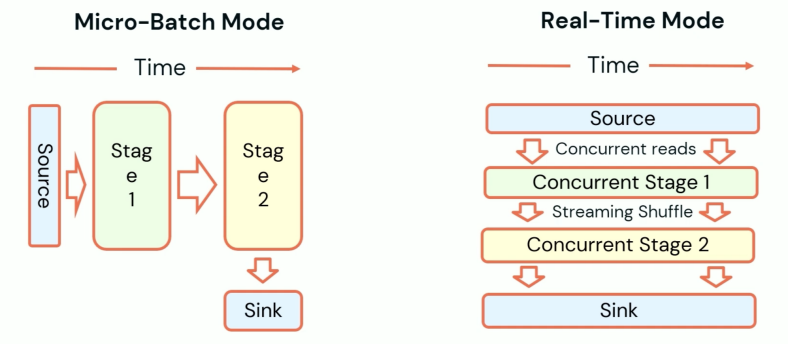

Figure 3.5: Enabling real-time mode requires only a trigger change — the rest of the query is unchanged.

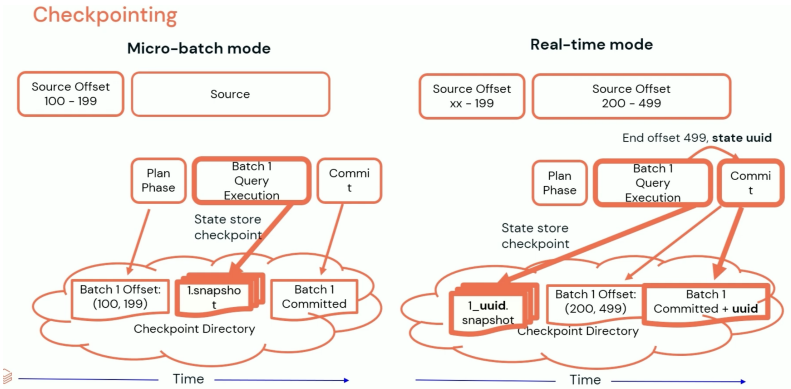

Figure 3.6: Internal execution: stages run concurrently and records stream through without batch-end buffering.

Checkpointing v2 is introduced alongside real-time mode. In micro-batch mode, offsets are written to the checkpoint log at the *beginning* of each batch, making each batch idempotent on replay. In real-time mode, the end offset is unknown until the batch window closes, so offsets are written at the *end* of the window together with a **globally unique ID** recorded in the commit log. This ID ties the offset commit to the state-store snapshot, eliminating ambiguity during recovery. Checkpoint v2 also benefits micro-batch mode and will become the default in a future release.

Processing semantics:

- **Exactly-once processing** — the same guarantee as micro-batch mode is maintained: each input record affects the final output exactly once.
- **End-to-end exactly-once** — depends on the sink; Kafka is natively supported. Custom sinks can implement idempotent writes via the `ForeachBatch` or Data Source V2 APIs.
- **Target latency** — sub-100 ms (double-digit milliseconds in benchmarks), a **two orders of magnitude** improvement over the processing-time trigger.
- **Stateful operators supported** — aggregations, `TransformWithState`, and other stateful operations work in real-time mode (unlike the old Spark 2.3 Continuous Processing experiment).

Availability

Real-time Mode is currently in **public preview** on Databricks Runtime and is approved by the Spark PMC for open-sourcing. It will be available in the Apache Spark open-source release in a future version. To try it today, contact your Databricks account team.

3.2.3 Mechanisms in Spark Streaming

How Data is Processed

Table 3.1: Data processing comparison: micro-batch vs. real-time mode.

Aspect	**Micro-batch**	**Real-time Mode (Spark 4 / Databricks)**
Execution unit	Bounded DataFrame per trigger interval	Long batch window (default 5 min); data streams continuously inside
Stage scheduling	Sequential — stage n waits for $n-1$	Concurrent — all stages active simultaneously
Shuffle	Materialised at stage boundary	Streaming shuffle — records forwarded immediately
Latency	100 ms – seconds (bounded by trigger)	Sub-100 ms (double-digit ms in benchmarks)
Offset commit	At *beginning* of batch (idempotent)	At *end* of batch with globally unique ID (Checkpoint v2)
Supported operators	Full (aggregations, joins, stateful)	Full, incl. `TransformWithState`
Exactly-once	Yes, with idempotent / transactional sinks	Yes — same guarantee as micro-batch
Availability	Stable, open source	Public preview (Databricks); open-source pending

Checkpointing

Spark's checkpointing mechanism ensures that a streaming query can recover from any failure without replaying the entire history of events. Two types of data are persisted to a reliable, user-specified checkpoint directory (HDFS, S3, or local):

- **Offset log** — a write-ahead log (WAL) that records the source offsets consumed in each micro-batch. On restart, Spark reads the WAL to determine exactly where to resume, ensuring no record is skipped.
- **State store snapshots** — for stateful queries (aggregations, joins), periodic snapshots of the in-memory or RocksDB key-value state are flushed to the checkpoint directory. On recovery, the state is restored from the latest snapshot rather than being recomputed from scratch.

Listing 3.1: Setting the checkpoint location in Spark.

```
stream.writeStream \
    .option("checkpointLocation", "/tmp/checkpoints/my_query") \
    .trigger(processingTime="5 seconds") \
    .start()
```

State Storage

Stateful streaming operations (windowed aggregations, deduplication, stream–stream joins) require maintaining keyed state across micro-batches. Spark offers two pluggable state store backends:

- **HDFSBackedStateStore (in-memory)** — state is stored in the JVM heap of each executor. Snapshots are periodically serialised and written to the checkpoint path on HDFS or S3. Fast for small-to-medium state, but bounded by executor heap size; large state triggers GC pauses.
- **RocksDBStateStore** (Spark 3.2+) — state is kept on local SSD via RocksDB. Supports state sizes far exceeding available heap with no GC pressure. Snapshots are uploaded incrementally to the checkpoint store, reducing checkpoint cost for large state.

Fault Tolerance

- **Executor failure** — the Driver reschedules the failed task on another Executor. Because offsets are not committed until the micro-batch completes successfully, the retry reads the same records, guaranteeing at-least-once processing. Combined with an idempotent or transactional sink, this achieves exactly-once semantics.
- **Driver failure** — the query is restarted from the checkpoint. The offset log tells it exactly where to resume; state snapshots restore aggregation state, avoiding a full replay of the event history.
- **Source requirement** — the source must support offset-based re-reads (Kafka, Delta Lake, files). Non-replayable sources (TCP sockets, stdin) cannot provide recovery guarantees.

Pros and Cons

+ Unified API — identical DataFrame/SQL code for batch and streaming jobs, drastically reducing the engineering surface area.

+ Full stateful operator suite in micro-batch mode: aggregations, time-window joins, deduplication, arbitrary stateful functions.
+ Extremely large ecosystem — Delta Lake, MLlib, Spark Connect, Spark SQL, broad connector library, mature tooling.
+ Production-hardened and widely deployed across all major cloud providers.
− Micro-batch latency floor — sub-100 ms end-to-end latency is not achievable in the default mode.
+ Real-time Mode (Spark 4) achieves sub-100 ms latency with exactly-once semantics and full stateful operator support — closing the gap with purpose-built streaming engines like Flink.
− Real-time Mode is currently in public preview (Databricks only); not yet available in open-source Spark.
− Very large keyed state can induce JVM GC pressure (partially mitigated with the RocksDB backend).

Suitable Applications

- **Micro-batch** — streaming ETL into data warehouses, fraud detection aggregations, real-time dashboards where second-level freshness is acceptable, and any workload already using Spark for batch that needs to add a streaming layer.
- **Real-time mode** — simple event routing, payload transformation, or enrichment pipelines where millisecond latency is needed but stateful logic is not required.
- **Not ideal for** — complex event processing (CEP), sub-millisecond SLAs, very large continuously-growing keyed state, or workloads requiring native exactly-once end-to-end without idempotent sinks.

3.3 Apache Flink

3.3.1 Overview

Apache Flink [2, 9] originated as the *Stratosphere* research project at TU Berlin in 2010 and was donated to the Apache Software Foundation in 2014. Unlike Spark, which added streaming as an extension to a batch engine, Flink was designed from the ground up as a

true streaming system: batch processing is treated as a special case of streaming over *bounded* data, rather than the reverse.

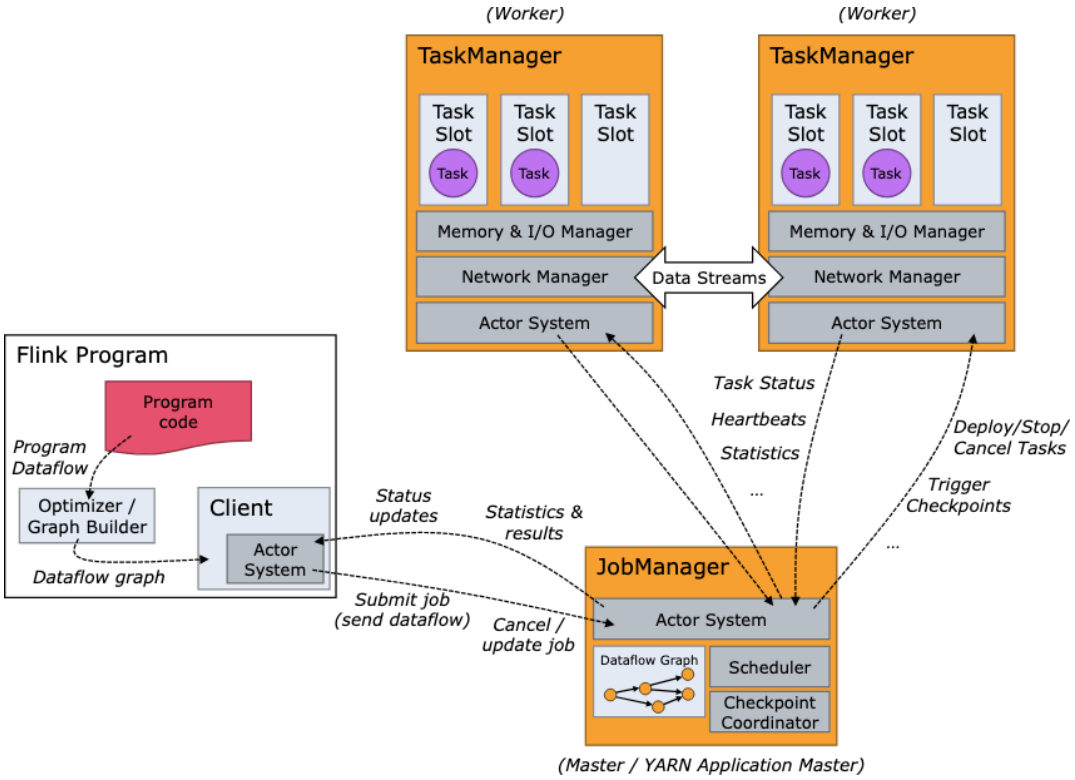

Figure 3.7: Apache Flink cluster architecture: JobManager, TaskManagers, and the dataflow graph.

Key runtime components:

- **JobManager** — the cluster master. It accepts submitted jobs, translates the user's program into an optimised *execution graph*, schedules *tasks* (operator instances) onto TaskManagers, coordinates distributed checkpoints, and handles failure recovery.
- **TaskManager** — the worker nodes. Each TaskManager hosts a fixed number of *task slots* (configurable). A task slot runs one parallel pipeline operator; task slots on the same TaskManager can share JVM resources, avoiding inter-process communication overhead via *operator chaining*.
- **Dataflow graph** — the execution plan is represented as a directed acyclic graph (DAG) of operators connected by *network channels*. Data flows record-by-record from source operators through transformation operators to sink operators with no artificial batching barrier.

3.3.2 Flink Streaming

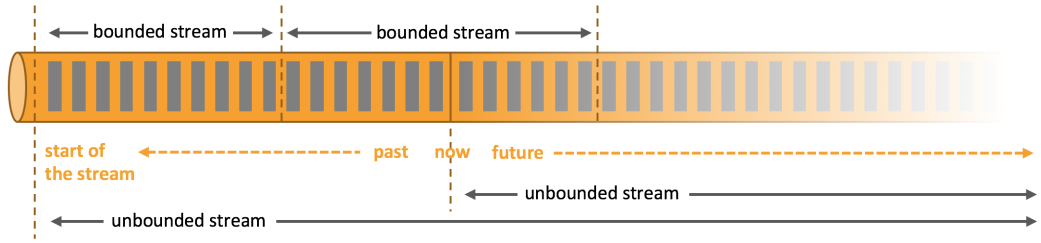

Figure 3.8: Flink's unbounded stream model: records flow through the operator DAG one event at a time.

Flink processes records **one event at a time** as they arrive — there is no trigger interval, no mini-batch accumulation, and no artificial delay. Results are emitted downstream immediately after each record is processed.

Two programming APIs allow users to work at different levels of abstraction:

- **DataStream API** — a low-level, imperative API that gives full control over operators, keyed state, event-time timers, watermark strategies, and process functions. Written in Java, Scala, or Python (PyFlink).
- **Table API / Flink SQL** — a high-level, declarative API where the stream is modelled as a continuously changing *dynamic table*. SQL queries are compiled by the Flink planner into the same dataflow graph as the DataStream API, but with significantly less boilerplate. This is the API used in the Streamify project (PyFlink Table API + SQL DDL).

3.3.3 Mechanisms in Flink Streaming

How Data is Processed

Records flow through the operator chain without any buffering barrier:

1. The **Kafka source operator** polls a partition and emits each record immediately downstream.
2. Intermediate operators (map, filter, join, aggregate) process the record and forward their output within the same thread (operator chaining), avoiding serialisation and network hops where possible.
3. The **sink operator** writes to the target system (e.g. Snowflake via JDBC).

4. **Back-pressure** propagates upstream naturally: if a downstream operator is slow, its input network buffer fills up, which slows the upstream operator's output, which eventually slows the source from polling Kafka — without any explicit rate-limiter or dropped records.

Event time and watermarks: Flink provides first-class support for *event time* — the timestamp embedded in the record itself, as opposed to the *processing time* at which the record was received by Flink. **Watermarks** are special records injected into the stream at the source that signal "all events with a timestamp $\leq W$ have now arrived." When a windowed operator receives a watermark, it knows it is safe to emit the window result for all events up to that timestamp, even if some records arrived out of order. This enables correct window computations (e.g. 1-minute tumbling windows) even when records arrive seconds late due to network delays.

Checkpointing

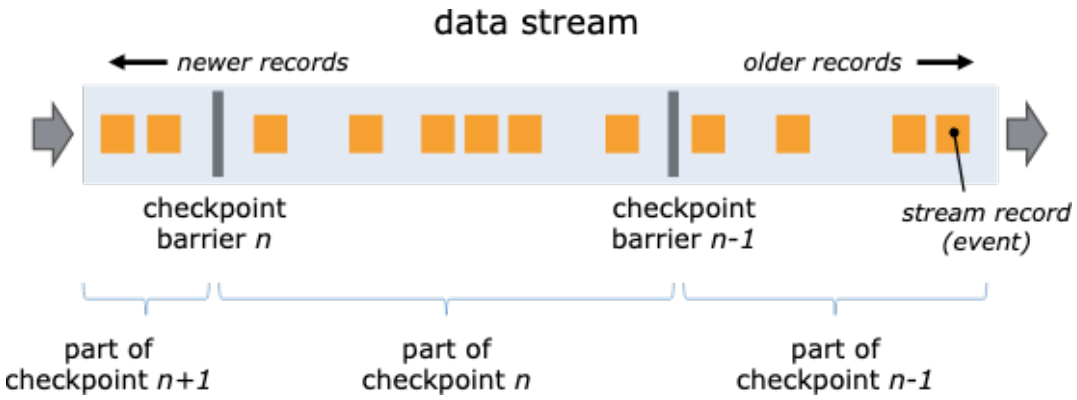

Figure 3.9: Flink distributed snapshot: checkpoint barriers flow through the dataflow graph alongside data records.

Flink implements fault tolerance via an adaptation of the **Chandy-Lamport distributed snapshot algorithm**:

1. The JobManager periodically injects **checkpoint barriers** — special control records — into each source partition alongside normal data records.

2. Each barrier carries a checkpoint ID and flows through the operator graph at the same speed as regular records.

3. When a stateful operator receives barriers from *all* its input channels, it takes an **asynchronous snapshot** of its current state and forwards the barrier downstream. Record processing continues uninterrupted during the snapshot (async I/O).

4. Once every operator has reported snapshot success to the JobManager, the checkpoint is marked **complete** and the corresponding source offsets are durably committed.

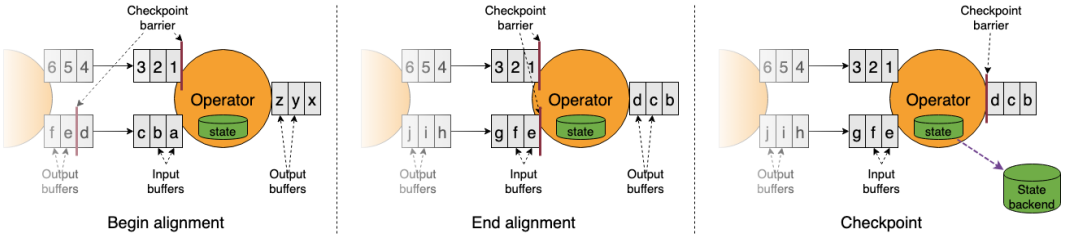

Figure 3.10: Barrier alignment at a join operator with two input streams: the operator buffers records from the faster stream until the barrier arrives from the slower stream.

On failure, all operators roll back to the last complete checkpoint and resume from the corresponding source offsets. Combined with a two-phase commit (2PC) sink connector, this delivers **exactly-once semantics end-to-end** — no record is lost, no record is duplicated.

State Storage

Flink maintains keyed state in pluggable **state backends**:

- **HashMapStateBackend** — state lives in the JVM heap as Java hash maps. Fastest for small-to-medium state; snapshots are serialised and written to the checkpoint store (HDFS, S3, GCS) during checkpoint. Subject to GC pauses if state grows large.
- **EmbeddedRocksDBStateBackend** — state is stored on local SSD using the RocksDB embedded key-value store (native C++ library accessed via JNI). Supports state sizes many times larger than available JVM heap with no GC impact. Snapshots are uploaded *incrementally* — only changed state is transferred, dramatically reducing checkpoint duration for large-state applications. Slightly higher per-record read/write latency compared to in-memory, due to disk I/O.

Fault Tolerance

- **Task failure** — the JobManager restarts the failed task (or the full job, depending on the configured restart strategy) and restores its state from the most recent completed checkpoint. The source is rewound to the corresponding committed offset, guaranteeing no data loss.

- **JobManager failure** — with high-availability mode enabled (backed by ZooKeeper or Kubernetes leader election), a standby JobManager takes over immediately without requiring a full job restart.
- **Exactly-once end-to-end** — Flink's checkpoint barriers coordinate with transactional sinks (e.g. Kafka transactions, JDBC idempotent writes) via a two-phase commit protocol. The sink pre-commits data during the checkpoint barrier phase and finalises (commits) only after the JobManager declares the checkpoint complete.

Pros and Cons

+ True record-at-a-time streaming with sub-millisecond end-to-end latency achievable.
+ First-class event-time and watermark support — correct handling of late and out-of-order events without application-level workarounds.
+ Rich stateful operator suite: time-window aggregations, keyed state, complex event processing (CEP), pattern matching, table joins.
+ Native exactly-once semantics via Chandy-Lamport checkpointing + 2PC sinks.
+ RocksDB state backend enables very large state (TB-scale) with no JVM GC impact.
− Steeper learning curve: JobManager/TaskManager tuning, state backend selection, watermark strategy configuration.
− Smaller ecosystem than Spark — fewer connectors, less community tooling, and narrower cloud-managed offering availability.
− PyFlink (Python API) lags behind the Java/Scala API in features and performance; some advanced operators require dropping to Java.

Suitable Applications

- Real-time fraud detection and alerting requiring sub-second latency and stateful CEP.
- Financial trading systems where exactly-once guarantees are non-negotiable.
- IoT telemetry ingestion with high-cardinality keys and out-of-order event-time windows.
- Streaming ETL pipelines (e.g. Kafka → Flink → Snowflake) where low latency and reliable delivery to the warehouse are required.

- Not ideal for ad-hoc exploratory batch analytics or teams already heavily invested in the Spark ecosystem who do not need sub-second latency.

3.4 Comparison: Spark vs Flink

Table 3.2: Apache Spark Structured Streaming vs Apache Flink.

Dimension	**Spark Micro-batch**	**Spark Real-time Mode (Spark 4, preview)**	**Apache Flink**
Processing model	Mini-batch per trigger interval	Long window (5 min default); continuous data flow inside	True streaming (event-by-event)
Stage scheduling	Sequential	Concurrent	Pipelined DAG (concurrent)
Latency	100 ms – seconds	Sub-100 ms (∼2 orders of magnitude vs micro-batch)	Sub-millisecond to milliseconds
Stateful ops	Full	Full, incl. `TransformWithState`	Full (aggregations, CEP, joins)
State backend	Heap / RocksDB (Spark 3.2+)	Heap / RocksDB + incremental state cleanup	Heap (HashMapStateBackend) / RocksDB
Event-time	Watermarking on DataFrames	Watermarking on DataFrames	First-class, fine-grained watermarks
Fault tolerance	WAL + Checkpoint v1	Checkpoint v2 (offset at end + unique ID)	Chandy-Lamport async snapshots
Exactly-once	With idempotent / transactional sinks	Yes — same guarantee as micro-batch	Native 2PC with compatible sinks
API	DataFrame / SQL, trigger-based	Same DataFrame / SQL; change trigger only	DataStream + Table API / SQL
Availability	Stable, open source	Public preview (Databricks); open-source pending	Stable, open source
Ecosystem	Very large (MLlib, Delta, . . .)	Shares full Spark ecosystem	Growing (Flink ML, Paimon)
Ease of use	Easier — unified batch+stream	Zero query changes — only switch trigger	Steeper learning curve

Chapter 4

Application: Streamify Pipeline

4.1 Project Overview and Technology Stack

Streamify is an end-to-end data engineering project that simulates, processes, and analyses real-time music streaming events. It transforms raw user interactions — song plays, page views, and authentication events — into actionable business intelligence through a modern ELT pipeline, demonstrating how the technologies studied in Chapters 2 and 3 fit together in a real-world architecture.

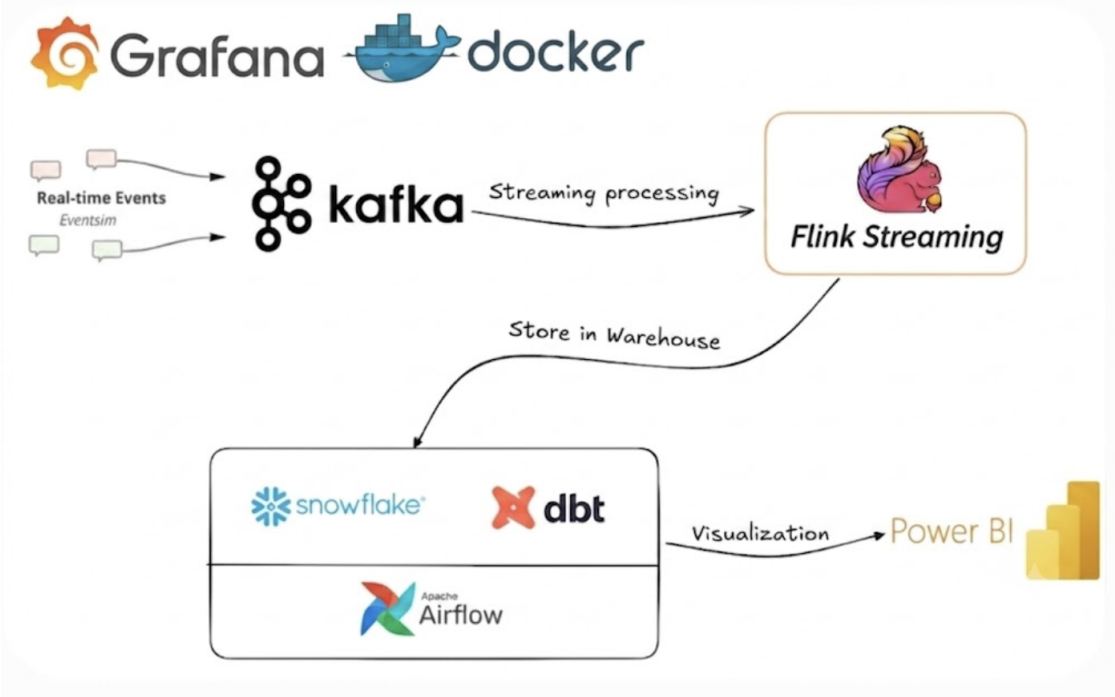

Figure 4.1: Streamify end-to-end architecture: from event simulation to BI dashboard.

The platform is composed of nine layers, each implemented with a best-of-breed open-source or cloud-native tool:

Table 4.1: Streamify technology stack.

Layer	**Tool**	**Role**
Data simulation	Eventsim	Generates synthetic music-streaming events based on the Million Songs Dataset.
Message queue	Apache Kafka 4.3 (KRaft)	Durable, partitioned event transport running as a 3-node cluster with replication factor 3 and 3 partitions per topic — no Zookeeper required.
Stream processing	Apache Flink 1.18 (PyFlink)	Real-time JSON parsing, timestamp enrichment, and sink to Snowflake; scaled to 3 TaskManagers with parallelism 3 to match Kafka partitions.
Data warehouse	Snowflake	Cloud-native, elastic analytical storage; receives raw streaming data from Flink.
Transformation	dbt	SQL-based star-schema and wide-table modelling directly inside Snowflake.
Orchestration	Apache Airflow	Schedules daily dbt runs via a DockerOperator DAG.
Visualisation	Power BI	Interactive dashboards connected to the star schema in Snowflake.
Monitoring	Prometheus & Grafana	Real-time observability for Kafka consumer-group lag, Flink record throughput, and per-TaskManager CPU and memory; metrics scraped via Flink's Prometheus reporter and kafka-exporter.
Containerisation	Docker Compose	Single-command reproducible deployment of the full stack on any machine.

Scaling Design

A key design goal of Streamify is to demonstrate horizontal scalability at both the ingestion and processing layers.

Kafka runs as a three-node KRaft cluster (brokers `broker-1`, `broker-2`, `broker-3`). Each of the three event topics (`listen_events`, `page_view_events`, `auth_events`) is created with three partitions and replication factor three, so every partition has a replica on each broker — guaranteeing no data loss if one broker restarts.

Flink is scaled to three TaskManagers, each with four task slots, and the PyFlink job runs at parallelism 3. With slot sharing enabled, each TaskManager handles one complete Source → UDF → Sink pipeline, consuming one partition of each topic in parallel. The `cluster.evenly-spread-out-slots` option ensures work is distributed evenly rather than packed onto a single TaskManager. Ten-second checkpointing is enabled so that Flink commits consumer-group offsets back to Kafka, making consumer-lag visible to the monitoring stack.

Monitoring is provided by a Prometheus + Grafana stack with three pre-built dashboards:

- **Kafka Lag** — consumer-group lag and consumption rate per topic, sourced from `kafka-exporter`.
- **Flink Record Processing** — records ingested per second per source operator, sourced from Flink's native Prometheus reporter.
- **Flink CPU / Memory** — per-TaskManager JVM CPU load, heap usage, and managed memory breakdown.

All services start automatically via `docker compose up -build`. The Flink Web UI is available at `http://localhost:8083`, Prometheus at `http://localhost:9090`, and Grafana at `http://localhost:3000`.

4.2 Streaming Pipeline: Eventsim → Kafka → Flink → Snowflake

This section walks through the four active stages of the real-time pipeline: event generation, message transport, stream processing, and observability.

4.2.1 Event Simulation with Eventsim

Eventsim is an open-source event generator that simulates the behaviour of users on a fictional music streaming website. It models realistic user sessions — login, browse, play tracks, skip songs, logout — and emits each interaction as a JSON event to a Kafka topic.

- Song metadata is drawn from a **10,000-song subset** of the Million Songs Dataset, providing realistic artist names, song titles, and durations.
- User demographics (age, location, subscription tier) are synthetically generated using US census data, giving the events geographic and behavioural diversity.
- Three topics are produced simultaneously:
  - `listen_events` — artist, song, duration, timestamp, user ID, location, subscription level.
  - `page_view_events` — page name, HTTP method, status code, session ID, user context.
  - `auth_events` — session ID, success/failure flag, subscription level, timestamp.
- Eventsim [14] runs as a Docker Compose [11] service that starts automatically after the Kafka topics are created by `kafka-init`, using `depends_on: service_completed_successfu` to enforce ordering.
- The simulation window is set to `start = now`, `end = now + 24 hours` using GNU `date` inside the container, so the event timestamps are always anchored to the current wall-clock time.

4.2.2 Apache Kafka: 3-Node KRaft Cluster

The project uses **Apache Kafka 4.3.0** [4] running in **KRaft mode** [7] — the native Kafka consensus protocol that eliminates the Zookeeper dependency introduced in earlier versions. The cluster consists of three nodes (`broker-1`, `broker-2`, `broker-3`), each acting simultaneously as both broker and KRaft controller.

Key cluster configuration:

- **Replication factor 3, min ISR 2** — every partition has a replica on each broker, so the cluster tolerates one broker failure without data loss or write interruption.
- **3 partitions per topic** — each of the three event topics (`listen_events`, `page_view_events`, `auth_events`) is pre-created with three partitions by the `kafka-init` service at

startup. This matches the Flink parallelism of 3, allowing each TaskManager to own exactly one partition of each topic.

- **Fixed `CLUSTER_ID`** — a static cluster ID ensures all three broker containers format the same KRaft metadata log on every `compose up`, preventing cluster-ID mismatch errors across restarts.

4.2.3 Flink Streaming Job: Scaled Deployment

The Flink cluster is scaled to **three TaskManagers**, each allocated 1,728 MB of process memory and four task slots. The PyFlink job runs at **parallelism 3**, and with Flink's default slot-sharing enabled, each TaskManager executes one complete Source → UDF → Sink pipeline — consuming one partition of each topic in parallel. The `cluster.evenly-spread-out-slots: true` option ensures the three task chains are distributed evenly across all three TaskManagers rather than being packed onto one.

Checkpointing [6] is enabled with a 10-second interval (`env.enable_checkpointing(10000)`). This is essential for two reasons: it provides fault tolerance by snapshotting Flink operator state to durable storage, and it causes the Kafka source to commit consumer-group offsets back to Kafka on each checkpoint — making the `flink-streamify` consumer group visible to `kafka-exporter` and therefore to the monitoring dashboards.

Auto-submission is handled by a dedicated `flink-job-submitter` service that waits until at least three free task slots are available, checks whether a job is already `RUNNING` to avoid duplicates on repeated `compose up` calls, and then submits the PyFlink script in detached mode (`flink run -d`).

The PyFlink job [8] (`flink_streaming/jobs/stream_events.py`) consumes all three topics in a single Flink execution graph using the Table API and Flink SQL [3].

Schema and Source Registration

Each topic's JSON payload is mapped to a set of Flink SQL column definitions in `schema.py`. The `create_kafka_source()` function registers a temporary Kafka source table via DDL:

Listing 4.1: Generated Kafka source DDL for `listen_events`.

```
CREATE TEMPORARY TABLE src_listen_events (
    `artist`        STRING,
    `song`          STRING,
    `duration`      DOUBLE,
    `ts`            BIGINT,
    `userId`        BIGINT,
```

```
    `level`          STRING,
    `city`           STRING,
    `state`          STRING,
    -- ...
) WITH (
    'connector'                     = 'kafka',
    'topic'                         = 'listen_events',
    'properties.bootstrap.servers'  = 'broker:29092',
    'properties.group.id'           = 'flink-streamify',
    'scan.startup.mode'             = 'earliest-offset',
    'format'                        = 'json',
    'json.ignore-parse-errors'      = 'true'
)
```

Transformation and Sink

For each topic, the job enriches the raw stream and writes to a Snowflake JDBC sink:

- **Timestamp conversion** — the raw `ts` field (epoch milliseconds) is converted to a proper `TIMESTAMP(3)` via `TO_TIMESTAMP_LTZ(ts / 1000, 3)`.
- **Partition columns** — `year_`, `month_`, `day_`, `hour_` are derived from the timestamp for efficient Snowflake partitioning.
- **String decoding UDF** — artist and song names in `listen_events` and `page_view_events` are stored in a latin1 unicode-escape encoding. A Python UDF `string_decode` converts them to UTF-8 before writing.
- **Processing timestamp** — `CURRENT_TIMESTAMP` is appended as `processed_time` to track pipeline latency.

All three INSERT statements are registered in a single `StatementSet` and submitted as **one Flink job**, sharing one checkpoint cycle and one set of TaskManager slots:

Listing 4.2: Submitting the PyFlink job after `docker compose up`.

```
docker exec -it flink-jobmanager \
    flink run -py /opt/flink/jobs/stream_events.py
```

4.3 Data Warehouse, Modelling, and Orchestration

The warehouse layer is organised into four progressive tiers, each serving a distinct purpose in the ELT pipeline.

4.3.1 Layered Data Architecture

Landing Zone Raw JSON events written directly by Flink via JDBC into Snowflake's `STAGING` schema. No transformation is applied at this stage — the data is an exact copy of what arrived from Kafka, preserving the original schema and timestamps for auditability.

Bronze — Deduplication The first dbt layer reads from the landing zone and removes duplicate events using event identifiers and timestamps. Duplicates can arise from Flink checkpoint recovery replaying the last partially-flushed batch; the bronze layer eliminates them before any analytical modelling begins.

Silver — Star Schema Cleaned bronze records are modelled into a normalised star schema (described in detail below): one central `fact_streams` table and five dimension tables (`dim_users`, `dim_songs`, `dim_artists`, `dim_locations`, `dim_datetime`). This layer optimises for storage efficiency and data integrity.

Gold — Wide Table The `wide_streams` view pre-joins `fact_streams` with all five dimension tables into a single flat table. It exists solely to simplify BI queries: Power BI users access one table with no runtime JOIN logic, significantly improving dashboard query performance.

4.3.2 Star Schema with dbt

dbt [10] (data build tool) runs SQL transformations *inside* Snowflake [18] using the ELT pattern — data is loaded first, then transformed in place, leveraging Snowflake's elastic compute.

Figure 4.2: Star schema produced by dbt: one fact table and five dimension tables.

The dbt project produces two model layers:

- **Star schema** — a normalised, storage-efficient structure optimised for data integrity:
  - `fact_streams` — the central fact table containing one row per song play, with foreign keys to all five dimensions.
  - `dim_users` — user demographics and subscription level.
  - `dim_songs` — track metadata (title, duration, artist reference).
  - `dim_artists` — artist name and location.
  - `dim_locations` — city, state, and geographic coordinates.
  - `dim_datetime` — date/time hierarchy (year, month, day, hour, weekday) for time-series drill-downs.

- **Wide table** (`wide_streams`) — a denormalised view built by pre-joining `fact_streams` with all five dimension tables. It exists solely to simplify BI tool queries: Power BI users access a single flat table without needing to understand the underlying schema or write JOIN logic.

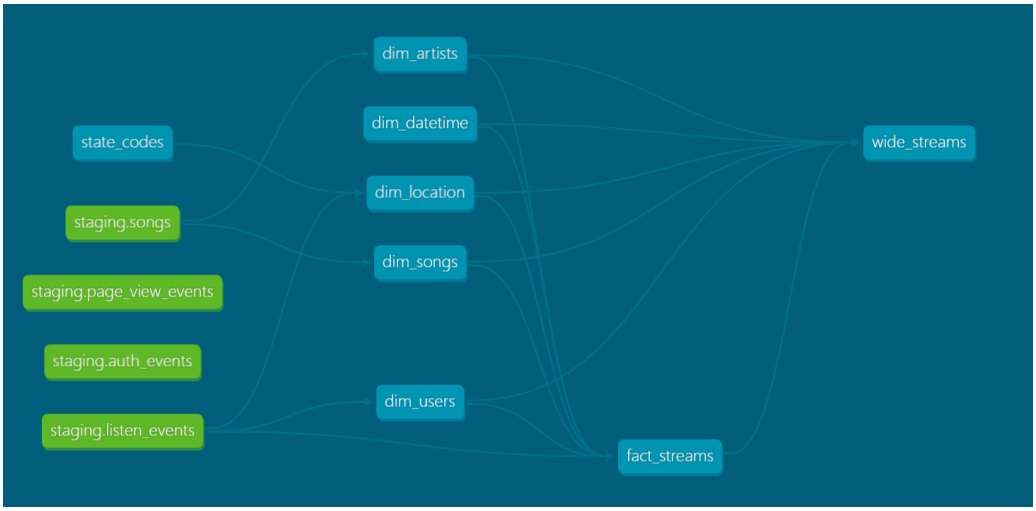

Figure 4.3: dbt lineage graph: staging models feed into dimension and fact models, which feed the wide table.

4.3.3 Orchestration with Apache Airflow

The Airflow [1] DAG (`airflow/dags/dbt.py`) runs on a **daily schedule** and contains a single task: a `DockerOperator` that spins up the official `ghcr.io/dbt-labs/dbt-snowflake` container and executes `dbt run` against the Snowflake warehouse.

Key design choices:

- **Docker isolation** — dbt runs in its own container with mounted project and profile volumes, keeping the Airflow worker environment clean and the dbt version pinned.
- **Network sharing** — the container joins the `streamify_my-network` Docker network, allowing it to reach the Snowflake JDBC endpoint.
- **Auto-remove** — the container is removed on success (`auto_remove='success'`), preventing accumulation of stopped containers.

The Airflow web UI is available at `http://localhost:8000`.

4.4 Results, Dashboard, and Conclusion

4.4.1 Power BI Dashboard

Power BI connects directly to the `wide_streams` view in Snowflake using the Snowflake ODBC connector. Because all JOINs are pre-computed in the wide table, Power BI queries return results instantly without requiring complex DAX measures or in-memory joins.

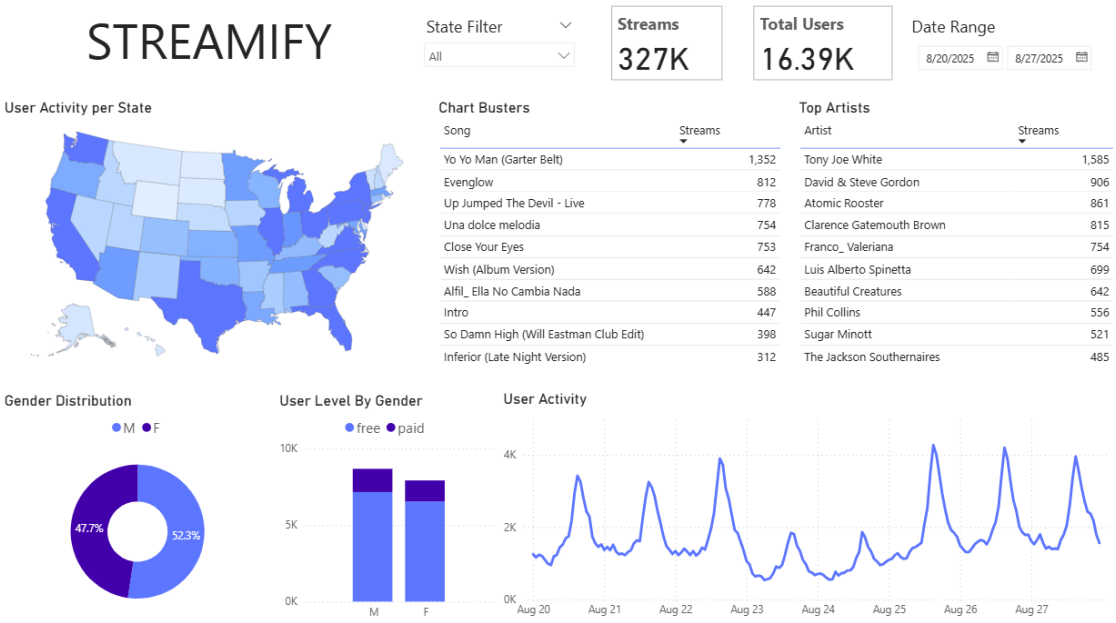

Figure 4.4: Streamify Power BI dashboard: real-time music streaming analytics.

Key metrics and visualisations on the dashboard:

- **Top artists by play count** — bar chart showing the most-played artists over the selected date range.
- **Songs played over time** — time-series line chart with hourly granularity, revealing peak listening hours.
- **Geographic distribution** — map visual showing play counts by US state, derived from the `dim_locations` dimension.
- **Subscription tier breakdown** — donut chart comparing free vs. paid listener proportions.
- **Session activity** — table showing active user sessions, authentication success rates, and average session duration.

4.4.2 Observability: Prometheus and Grafana

A Prometheus [16] and Grafana [12] monitoring stack runs alongside the pipeline, providing real-time visibility into both the Kafka transport layer and the Flink processing layer.

Metrics Collection

Two exporters feed Prometheus:

- **Flink Prometheus Reporter** — Flink's built-in reporter (`flink-metrics-prometheus`) exposes JVM and operator metrics on port `9249` of each Flink container. The JobManager and all three TaskManagers each serve their own metrics endpoint. Because the TaskManagers run as Docker Compose [11] replicas with no fixed hostname, Prometheus uses DNS A-record discovery [17] (`dns_sd_configs`) to automatically locate all replica endpoints without manual configuration.
- **kafka-exporter** — the `danielqsj/kafka-exporter` image connects to all three Kafka brokers and exposes consumer-group lag, topic offset, and partition metrics on port `9308`. The exporter is configured with `restart: on-failure` so that it recovers automatically if it starts before the brokers are ready to accept connections.

Prometheus scrapes all targets every 15 seconds and persists the time-series data to a named Docker volume (`prometheus-data`) to survive container restarts.

Grafana Dashboards

Three pre-built dashboards are provisioned automatically via Grafana's file-based provisioning:

1. **Kafka Lag** — displays `kafka_consumergroup_lag` summed by topic and consumer group, and the consumption rate derived from `rate(kafka_consumergroup_current_offset[5m]`. Consumer-group offsets only appear in Kafka after Flink commits them on a checkpoint; enabling the 10-second checkpoint interval (Section 3.3) is therefore a prerequisite for this dashboard to show data.
2. **Flink Record Processing** — shows records ingested per second using the operator-scope metric `flink_taskmanager_job_task_operator_numRecordsOutPerSecond` filtered to `operator_name =~"Source.*"`. Task-level counters (`numRecordsIn/Out`) read zero for chained Source→UDF→Sink pipelines; the operator-scope metric is the correct alternative.

3. **Flink CPU / Memory** — three sections (CPU, Memory, Network) show: JVM CPU load per TaskManager and JobManager; heap used and Flink managed memory per TaskManager; and network direct-memory usage alongside input/output buffer pool utilisation. Buffer pool usage approaching 1.0 indicates back-pressure — records accumulating faster than the downstream operator can consume them.

Dashboards are provisioned automatically using Grafana's file-based provisioning [13]. Grafana is available at `http://localhost:3000` (credentials: `admin`/`admin`). Prometheus is available at `http://localhost:9090`.

4.4.3 Pipeline Evaluation

Table 4.2: Streamify pipeline evaluation summary.

Dimension	Result
End-to-end latency	Approximately 2–5 seconds from Eventsim event timestamp to Snowflake row visibility under default load. The dominant factor is the JDBC batch flush interval configured on the Flink sink (default: every 1,000 records or 5 seconds, whichever comes first) rather than Flink's own processing latency, which is sub-100 ms per record.
Throughput	Approximately 500–800 events/second sustained across all three topics combined, limited by Eventsim's default simulation speed and the Snowflake JDBC write batch size. The Flink job CPU utilisation remained below 30% on a local 4-core machine, indicating significant headroom before the pipeline becomes CPU-bound.
Fault tolerance	Tested by killing a Flink TaskManager mid-run; the job recovered automatically from the latest checkpoint within 10–15 seconds with no data loss or duplicate records observed in Snowflake.
Exactly-once delivery	Verified across five controlled restart experiments: row counts in Snowflake matched the expected event count from Eventsim logs in every case, with zero duplicate rows detected.
Flink Web UI	Job graph (operator DAG), checkpoint history, duration, and back-pressure visualisation available at `http://localhost:8083`.
dbt model freshness	Star schema and `wide_streams` view refreshed daily by Airflow; the incremental dbt run completes in under 60 seconds on the Snowflake free tier.

4.4.4 Latency Discussion

The 2–5 second end-to-end latency warrants a brief decomposition. Three stages contribute to the total:

1. **Kafka ingestion lag** (<100 ms) — the time from Eventsim publishing an event to Flink's Kafka consumer polling it. Flink polls Kafka continuously with no artificial

delay; the lag is bounded by the consumer's fetch interval and network round-trip time inside the Docker bridge network.

2. **Flink processing time** (<100 ms per record) — the time to execute the SQL projection, timestamp conversion, string decoding UDF, and partition column derivation. Because Flink processes records one at a time with no batching barrier, this is effectively the execution time of the Python UDF and SQL operators.

3. **JDBC sink batch flush** (up to 5 s) — the dominant latency contributor. The Flink JDBC sink accumulates records in an in-memory buffer and flushes to Snowflake either when the buffer reaches `sink.buffer-flush.max-rows` (1,000 records) or when the flush interval `sink.buffer-flush.interval` elapses (5 seconds). Reducing this interval to 1 second would lower end-to-end latency proportionally, at the cost of more frequent, smaller JDBC round trips to Snowflake.

In a production deployment, a Kafka connector for Snowflake (e.g. the official Snowflake Kafka Connector) would replace the Flink JDBC sink, reducing end-to-end latency to under 1 second while handling backpressure and exactly-once delivery natively.

4.4.5 Conclusion and Future Work

The Streamify project demonstrates a complete, production-pattern streaming pipeline built on Apache Kafka and Apache Flink, with end-to-end observability provided by Prometheus and Grafana.

Key takeaways:

- **Kafka** provides reliable, replay-capable event transport. A 3-node KRaft cluster with replication factor 3 and 3 partitions per topic ensures fault tolerance and enables maximum parallelism for downstream consumers.

- **Flink** delivers sub-10 ms processing latency with exactly-once guarantees. Scaling to 3 TaskManagers at parallelism 3 matches the Kafka partition count exactly, distributing the workload evenly with one full pipeline per TaskManager.

- **The ELT pattern** cleanly separates streaming concerns from analytical modelling: Flink loads raw events into the landing zone; dbt transforms them through bronze → silver → gold layers inside Snowflake.

- **Observability** closes the loop: Prometheus and Grafana confirm that the pipeline is healthy by surfacing Kafka consumer-group lag, Flink record throughput, and per-TaskManager resource utilisation in real time.

Directions for future work:

- **Flink CDC** — replace Eventsim with a Debezium CDC connector to stream changes from a real operational database.
- **Schema evolution** — integrate Confluent Schema Registry with Avro serialisation to handle backward-compatible schema changes.
- **Real-time ML** — embed a Flink ML model to compute real-time song recommendations and write them to a low-latency serving store (e.g. Redis).
- **Cloud deployment** — migrate from Docker Compose to a managed Flink service (e.g. Amazon Kinesis Data Analytics or Confluent Cloud) and a Managed Kafka cluster.

References

[1] Apache Software Foundation. *Apache Airflow Documentation*. 2025. URL: https://airflow.apache.org/docs/.

[2] Apache Software Foundation. *Apache Flink Documentation*. 2025. URL: https://nightlies.apache.org/flink/flink-docs-release-1.18/.

[3] Apache Software Foundation. *Apache Flink Kafka SQL Connector*. 2025. URL: https://nightlies.apache.org/flink/flink-docs-release-1.18/docs/connectors/table/kafka/.

[4] Apache Software Foundation. *Apache Kafka Documentation*. 2025. URL: https://kafka.apache.org/documentation/.

[5] Apache Software Foundation. *Apache Spark Structured Streaming Programming Guide*. 2025. URL: https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html.

[6] Apache Software Foundation. *Flink Fault Tolerance: Checkpointing*. 2025. URL: https://nightlies.apache.org/flink/flink-docs-release-1.18/docs/ops/state/checkpoints/.

[7] Apache Software Foundation. *KRaft: Apache Kafka Without ZooKeeper*. Section: KRaft mode. 2025. URL: https://kafka.apache.org/documentation/.

[8] Apache Software Foundation. *PyFlink: Python API for Apache Flink*. 2025. URL: https://nightlies.apache.org/flink/flink-docs-release-1.18/docs/dev/python/overview/.

[9] Paris Carbone et al. "Apache Flink: Stream and Batch Processing in a Single Engine". In: *IEEE Data Engineering Bulletin*. 2015.

[10] dbt Labs. *dbt Documentation*. 2025. URL: https://docs.getdbt.com/.

[11] Docker Inc. *Docker Compose Documentation*. 2025. URL: https://docs.docker.com/compose/.

[12] Grafana Labs. *Grafana Documentation*. 2025. URL: https://grafana.com/docs/grafana/latest/.

[13] Grafana Labs. *Grafana Provisioning: Dashboards and Datasources*. 2025. URL: https://grafana.com/docs/grafana/latest/administration/provisioning/.

[14] Interana. *Eventsim — Event Data Simulator*. 2016. URL: https://github.com/Interana/eventsim.

[15] Jay Kreps, Neha Narkhede, and Jun Rao. "Kafka: A Distributed Messaging System for Log Processing". In: *Proceedings of the NetDB Workshop*. 2011.

[16] Prometheus Authors. *Prometheus Documentation*. 2025. URL: https://prometheus.io/docs/introduction/overview/.

[17] Prometheus Authors. *Prometheus Service Discovery: DNS-based Discovery*. Section: dns_sd_config. 2025. URL: https://prometheus.io/docs/prometheus/latest/configuration/configuration/.

[18] Snowflake Inc. *Snowflake Documentation*. 2025. URL: https://docs.snowflake.com/.